L Y N X F A C T O R Y

How your website goes from your laptop to the whole world

A complete walkthrough — assumes you've never heard of DNS, CDNs, webhooks or hashing.

Includes diagrams, plain-language explanations, troubleshooting, and a glossary.

Lynx Factory | GitHub | Cloudflare | Arsys

`edit → commit → push → build → serve`

Website deploy pipeline — English edition · v2.0
2026-05-17 · Borja Tarraso <borja.tarraso@member.fsf.org>

What is this document?

When you change a file inside one of your websites (galeriaomaso.com, enriquetahueso.com, etc.) several systems have to cooperate before that change appears in a stranger's browser. This guide explains every one of those steps in plain language.

It is written for two audiences. (1) The current operator who already runs Lynx Factory daily and wants the full picture of what is happening under the hood. (2) A newcomer who has just inherited this project, has never used Cloudflare or GitHub before, and needs to bring the whole stack up from a fresh machine.

How this guide is structured

- Orientation — what this document is, the four systems involved.
- Setup from a blank machine — accounts, scoped tokens, nameservers, Lynx install.
- How the pipeline works — diagrams, narratives, the Test button, the watcher.
- Day-to-day operations — adding sites, common commands, file tour.
- Failure analysis — why a previous version of the Test button silently failed, and what the new version does differently.
- Emergency procedures, troubleshooting, glossary, reference cards.

TIP

If a term confuses you, jump to the glossary near the end. Nothing here requires linear reading.

NOTE

Every command in this guide has been tested on Fedora Linux. They are POSIX shell commands and work on macOS and WSL with minor differences (package manager names).

The four systems involved

1. Lynx Factory — the orchestrator (lives on your laptop)

A Flask-based control room that you use to write, store, and operate every project you own. It is the only place you ever 'press a button' to deploy. Lynx talks to GitHub, Cloudflare, and your local files.

2. GitHub — version control (lives on github.com)

Stores every version of your website's source code. Pushing a new version to GitHub is the canonical signal that something has changed.

3. Cloudflare — build · cache · CDN (lives in 200+ cities)

Receives the new code (either via the wrangler CLI from your laptop, or via Cloudflare's GitHub auto-deploy integration), bundles it as a Worker with static assets, distributes it to edge POPs worldwide, and serves it. Cloudflare also acts as the authoritative DNS for the domain.

4. Arsys — your domain registrar (lives in Spain)

Owns the right to the name 'galeriaomaso.com' and similar. Arsys delegates resolution to Cloudflare's nameservers with one configuration setting. After that, Arsys is mostly out of the way until renewal time.

> **NOTE**
> Without Lynx → no convenient deploy. Without GitHub → no version history. Without Cloudflare → no hosting, caching or global delivery. Without Arsys → no domain name resolves to anything.

I just inherited this — where do I start?

Welcome. You are now responsible for a small fleet of websites and the tooling around them. Before you touch anything, follow this orientation:

Step 1 — Read the first two sections

You need a mental model of the four systems before you start changing things. It takes 5 minutes.

Step 2 — Get access to every account

- Cloudflare (dash.cloudflare.com) — hosting + CDN. Ask the previous operator for: account email, password (use a password manager), and 2FA backup codes.
- GitHub (github.com) — source code. You need either push access to the website repositories, or the org-admin role.
- Arsys (clientes.arsys.es) — domain registrar.
- Server / laptop where Lynx Factory runs — SSH or physical access.

Step 3 — Locate the existing config files

- web/data/websites.json — list of all websites Lynx Factory knows about.
- ~/.bashrc.d/lynx_factory_cf.sh — Cloudflare env vars (per-project scoped tokens, account ID, zone IDs).
- The Cloudflare API tokens themselves (in CF dashboard under My Profile → API Tokens).

Step 4 — Verify nothing is currently broken

```
# is Lynx Factory running?
ss -ltnp | grep 9999

# does the watcher answer?
curl -s http://127.0.0.1:9999/api/auto_redeploy_state | head -20

# do the live sites respond?
curl -sI https://www.galeriaomaso.com | head -1
curl -sI https://www.enriquetahueso.com | head -1
```

WARNING

Do not push any changes to the website repos until you can press the Test button in Lynx Factory and see it flash green. That confirms the whole pipeline (your laptop → wrangler/GitHub → Cloudflare → live) is healthy.

What you need installed + accounts to create

Software on your machine

- Python 3.11+ — runs Lynx Factory's Flask app and the watcher.
- git — version control.
- Node.js + npx — used by wrangler. The Test button invokes `npx wrangler@latest deploy`.
- A browser (Firefox/Chromium/Chrome/Brave/Edge/Vivaldi). The Test button uses one of these in incognito mode.
- An editor of your choice (VS Code, Emacs, Vim).
- Optional but recommended: ripgrep, jq, httpie for inspecting JSON and HTTP.

```
# Fedora / RHEL
sudo dnf install -y python3 python3-pip git nodejs npm ripgrep jq

# Debian / Ubuntu / WSL
sudo apt install -y python3 python3-pip git nodejs npm ripgrep jq

# macOS (with Homebrew)
brew install python git node ripgrep jq
```

Accounts you need

- Cloudflare (free tier is enough for static hosting) — sign up at dash.cloudflare.com.
- GitHub — sign up at github.com. Enable 2FA.
- Arsys (or whichever registrar holds your domains) — clientes.arsys.es.

Python packages used by Lynx Factory

Lynx Factory uses standard-library modules wherever possible. The few third-party packages it needs are:

```
pip install --user flask requests werkzeug

# For generating these docs:
pip install --user matplotlib pillow reportlab
```

NOTE

Lynx Factory deliberately avoids heavy frameworks. The Flask app, the watcher thread, and the JSON-on-disk data store keep everything inspectable with cat, grep, and curl.

Cloudflare account + least-privilege API tokens

1. Create a Cloudflare account

- Open dash.cloudflare.com → Sign up. Confirm your email.
- Turn on 2FA (My Profile → Authentication).

2. Add your zone (domain) to Cloudflare

- Dashboard → Add a site → enter yoursite.com → Free plan is fine.
- Cloudflare scans existing DNS records. Review them, click Continue.
- Cloudflare gives you two nameservers (e.g. alice.ns.cloudflare.com and bob.ns.cloudflare.com) — keep them for the Arsys step.

3. Find your Account ID and Zone ID

Any Cloudflare dashboard URL has the form dash.cloudflare.com/<ACCOUNT_ID>/<zone-name>/... — that's your Account ID. The Zone ID is shown on the zone Overview page (right sidebar).

```
# example only — replace with your real IDs
Account ID:  9f8b1c0e3d2f4e5a6b7c8d9e0f1a2b3c
Zone ID:     1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d
```

4. Create one scoped API token per project (least privilege)

Rather than one omnipotent token, mint one token per website with exactly the permissions that website's deploy needs:

- Name: token-scoped-refresh-<project> (e.g. token-scoped-refresh-galeriaomaso).
- Permissions: Account → Workers Scripts → Edit + Zone → Cache Purge → Purge.
- Account resources: Include → this account only.
- Zone resources: Include → Specific zone → exactly this site's zone.
- Continue → Create Token → copy the value once (you cannot see it again).

5. Store the tokens where Lynx will find them

```
# add to a file that .bashrc sources on every shell:
mkdir -p ~/.bashrc.d
cat > ~/.bashrc.d/lynx_factory_cf.sh <<'EOF'
export CF_ACCOUNT_ID="<your-account-id>"
export CLOUDFLARE_ACCOUNT_ID="$CF_ACCOUNT_ID"

export CF_API_TOKEN_GALERIAOMASO="<scoped-token-1>"
export CF_API_TOKEN_ENRIQUETAHUESO="<scoped-token-2>"

export CF_ZONE_ID_GALERIAOMASO="<zone-id-1>"
export CF_ZONE_ID_ENRIQUETAHUESO="<zone-id-2>"
EOF
chmod 600 ~/.bashrc.d/lynx_factory_cf.sh

# load now for this session:
source ~/.bashrc.d/lynx_factory_cf.sh
```

DANGER

The token file contains secrets. chmod 600 it. Never commit it to git. If a token leaks, revoke it from Cloudflare immediately and mint a fresh scoped one — never replace it with a more-powerful token.

TIP

Keep any 'powerful' (multi-project, broad-scope) token strictly under human control. Lynx Factory must always run on per-project scoped tokens.

Connecting Cloudflare to GitHub (one-time, optional)

Cloudflare can build and deploy automatically every time you push to GitHub. This is convenient but unreliable on its own — the integration can silently disconnect. Lynx Factory's wrangler-based path does not depend on it, but you may still want it enabled as a parallel route.

1. Create the Pages/Workers project

- Cloudflare Dashboard → Workers & Pages → Create application → connect to Git.
- Click Connect GitHub. A popup asks you to install the Cloudflare GitHub App.
- Authorize either 'All repositories' or the specific repo for this site.
- Return to the Cloudflare dashboard.

2. Configure the build

- Production branch: main
- Framework preset: 'None' for plain HTML, or pick your SPA framework.
- Build command: blank for static, otherwise e.g. 'npm run build'.
- Output directory: '/' for static, otherwise e.g. 'dist'.
- Click Save and Deploy.

3. Wire the custom domain

- Project → Custom domains → Set up a custom domain.
- Enter www.yoursite.com. Repeat for the apex.

4. Verify the GitHub webhook is live

- GitHub repo → Settings → Webhooks. Look for a 'cloudflare-pages' webhook with a green checkmark.
- Push a tiny commit; within seconds a new build should appear in CF Dashboard → Deployments.

WARNING

If pushes silently stop triggering deploys, this webhook is the first thing to suspect. Re-installing the Cloudflare GitHub App restores it. Meanwhile the Lynx Factory watcher + Test button bypass the webhook by calling wrangler directly.

Arsys nameserver setup (one-time per domain)

Arsys holds the domain registration but DNS resolution is delegated to Cloudflare. You do this once per domain.

1. Log in to Arsys

- Go to clientes.arsys.es and log in.
- Navigate to Mis productos → Dominios → click yoursite.com.

2. Replace the nameservers

- Find the 'Servidores DNS' / 'Nameservers' panel.
- Switch from 'DNS Arsys' to 'DNS externos'.
- Paste the two nameservers Cloudflare gave you.
- Save. Confirm propagation can take 24–48 hours.

3. Wait for propagation

```
# check from your laptop — should eventually show CF nameservers:
dig +short NS yoursite.com
# expected after propagation:
# alice.ns.cloudflare.com.
# bob.ns.cloudflare.com.
```

4. Confirm Cloudflare detects the change

On dash.cloudflare.com → Overview, the status banner switches from 'Pending Nameserver Update' to 'Active' (green). Propagation is usually a few hours, sometimes faster.

5. Keep auto-renew on at Arsys

Cloudflare handles DNS but Arsys still owns the domain. Enable auto-renew or keep a yearly calendar reminder. A lapsed domain takes the entire site down — neither Cloudflare nor Lynx can prevent it.

DANGER

Never switch nameservers back to 'DNS Arsys' unless you have moved hosting off Cloudflare. Doing so takes the site offline within minutes.

Lynx Factory — install + first boot

1. Clone the repository

```
git clone <repo-url> ~/claude/lynx_factory
cd ~/claude/lynx_factory
```

2. Install dependencies

```
pip install --user flask requests werkzeug matplotlib pillow reportlab
```

3. Source the env vars

```
source ~/.bashrc.d/lynx_factory_cf.sh
echo $CF_API_TOKEN_GALERIAOMASO | cut -c1-8   # sanity check
```

4. Boot Lynx Factory

```
# foreground (useful for the first run; shows logs):
./bin/lynx-factory --serve

# background (daemonized):
nohup ./bin/lynx-factory --serve > /tmp/lynx-factory.log 2>&1 &
disown

# open the UI in your default browser:
./bin/lynx-factory --browser
```

5. Verify it answers

```
ss -ltnp | grep 9999
curl -s http://127.0.0.1:9999/ | head -5
```

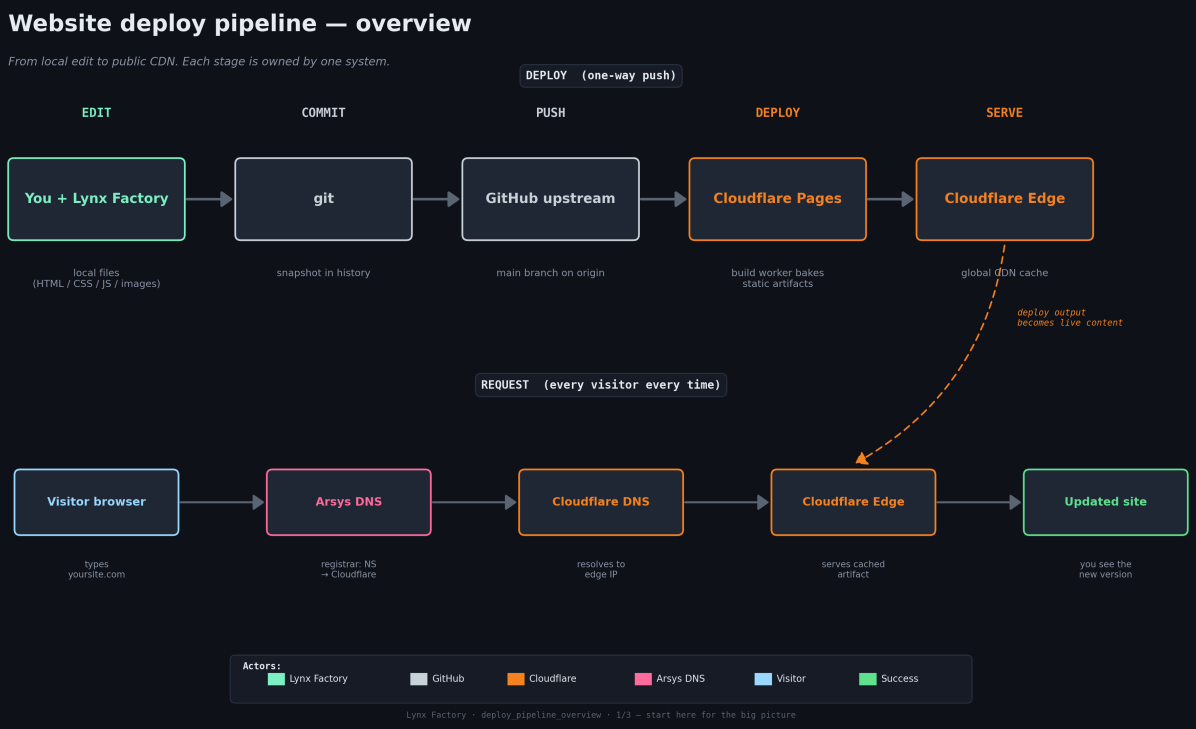
6. Open the multiplexer

Visit http://127.0.0.1:9999/multiplexer/web — you should see one cell per website with RST · EXEC · Test buttons across the top. Click Test on one and watch the verification flow run.

TIP

If Lynx Factory complains about missing env vars at startup, you forgot to source ~/.bashrc.d/lynx_factory_cf.sh in the same shell you launched it from. Open a fresh terminal and try again — .bashrc auto-sources that file.

Diagram 1 — Overview of the deploy + request flows



One change, end to end (story version)

Imagine you have just edited the banner image of galeriaomaso.com on your laptop. Here is exactly what happens, step by step, until a visitor in another country sees that new banner.

Step 1 — Save the file (your laptop)

You replace the old image with a new one. At this point nothing has reached the internet. The change exists only on your hard drive.

Step 2 — Make a git commit (your laptop)

git takes a snapshot: 'as of this moment, the banner file contains these bytes'. The snapshot lives in a hidden .git folder, still entirely local.

Step 3 — git push (laptop → GitHub)

You upload the new snapshot to GitHub. GitHub now has the same version you do.

Step 4 — Lynx Factory notices (within 60 seconds)

Lynx's watcher polls origin/main for every site. When it sees the SHA moved, it dispatches a deploy according to the site's deploy_method (typically wrangler).

Step 5 — wrangler deploys to Cloudflare

`npx wrangler@latest deploy` runs from the site's public/ directory, uploads the bundle, and Cloudflare publishes the new Worker version.

Step 6 — Cloudflare distributes to edge POPs

The new bundle propagates to 200+ data centers worldwide. The nearest POP serves each visitor from RAM in milliseconds.

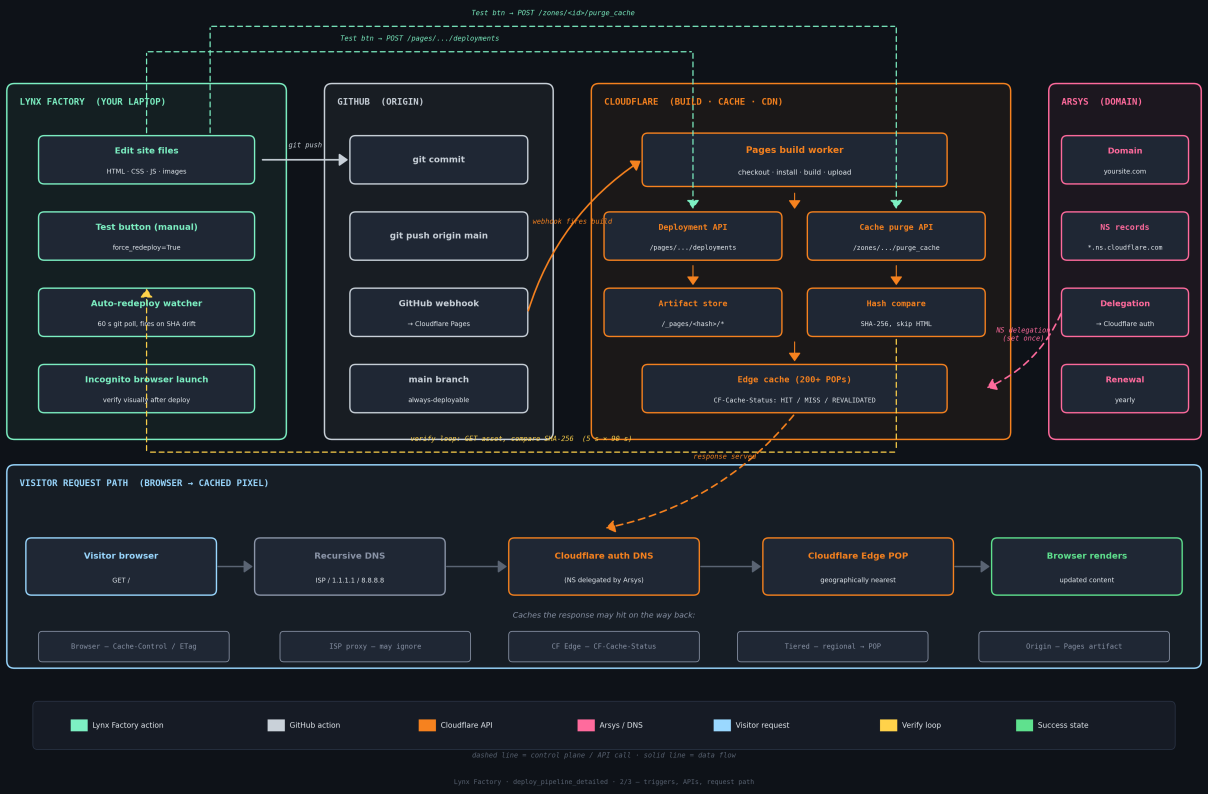
Step 7 — Verify loop confirms the edge is fresh

Lynx re-fetches the fingerprint asset, hashes it, and compares to local. When the SHA-256s match, the deploy is declared 'fresh'.

Step 8 — A visitor opens the page

Their browser resolves the domain via Arsys → Cloudflare DNS, routes to the nearest edge, and the new banner appears.

Diagram 2 — Detailed view (triggers, APIs, cache layers, DNS hops)



What the visitor actually experiences

DNS — turning a name into a number

Computers cannot talk to 'galeriaomaso.com' directly — they need an IP address. The Domain Name System translates one into the other. When a browser asks 'where is galeriaomaso.com?' the answer hops:

- Browser asks the recursive resolver (ISP, 1.1.1.1, 8.8.8.8).
- Resolver asks the top-level domain servers ('.com').
- TLD servers point at Arsys (because Arsys is the registrar).
- Arsys says 'this domain delegates to *.ns.cloudflare.com'.
- Resolver asks Cloudflare DNS for the IP.
- Cloudflare DNS replies with the IP of the nearest edge POP.

> **NOTE**
> After the first lookup the answer is cached at the resolver and the browser for some time. That is why DNS changes can take a while to spread.

CDN — many edge servers, one website

A Content Delivery Network is a global fleet of small data centers. Each stores a copy of your static files. The visitor in Tokyo hits the Tokyo edge; the visitor in Helsinki hits the Helsinki edge — but they see the same site.

Cache layers — where stale content hides

- Browser cache: the visitor's own browser keeps copies based on Cache-Control headers.
- ISP / corporate proxy: some networks cache responses between you and Cloudflare.
- Cloudflare edge cache: each edge POP keeps recent responses in RAM.
- Cloudflare tiered cache: a regional cache between edge POPs and the origin.
- Origin: the Worker bundle / static-asset store (source of truth).

> **WARNING**
> 'I changed the file but my phone still sees the old one' is almost always a cache problem at one of those five layers. The Test button purges Cloudflare's edge cache, but it cannot purge your phone's local cache — use an incognito tab.

The Test button — what it actually does

On every website cell in the multiplexer there is a small mint-teal Test button. Pressing it runs a single end-to-end health check that touches every step of the pipeline.

Step by step

- Read websites.json to find live_url, source_dir, public_subdir, fingerprint_asset, deploy_method and CF credentials.
- Optionally purge Cloudflare's cache for that zone (so the next fetch comes from origin).
- Download the fingerprint asset with a cache-busting query param.
- SHA-256 both the live copy and the local copy; compare.
- If hashes differ → dispatch a redeploy via _dispatch_deploy() (wrangler or pages_api).
- Verify loop: every 5s for up to 90s, refetch and rehash until they match.
- Probe a known new asset (skips config files like _headers, _redirects, _worker.js, wrangler.*).
- Check git: is the local clone dirty? Ahead/behind origin?
- Open the live URL in an incognito browser tab.

Click modifiers

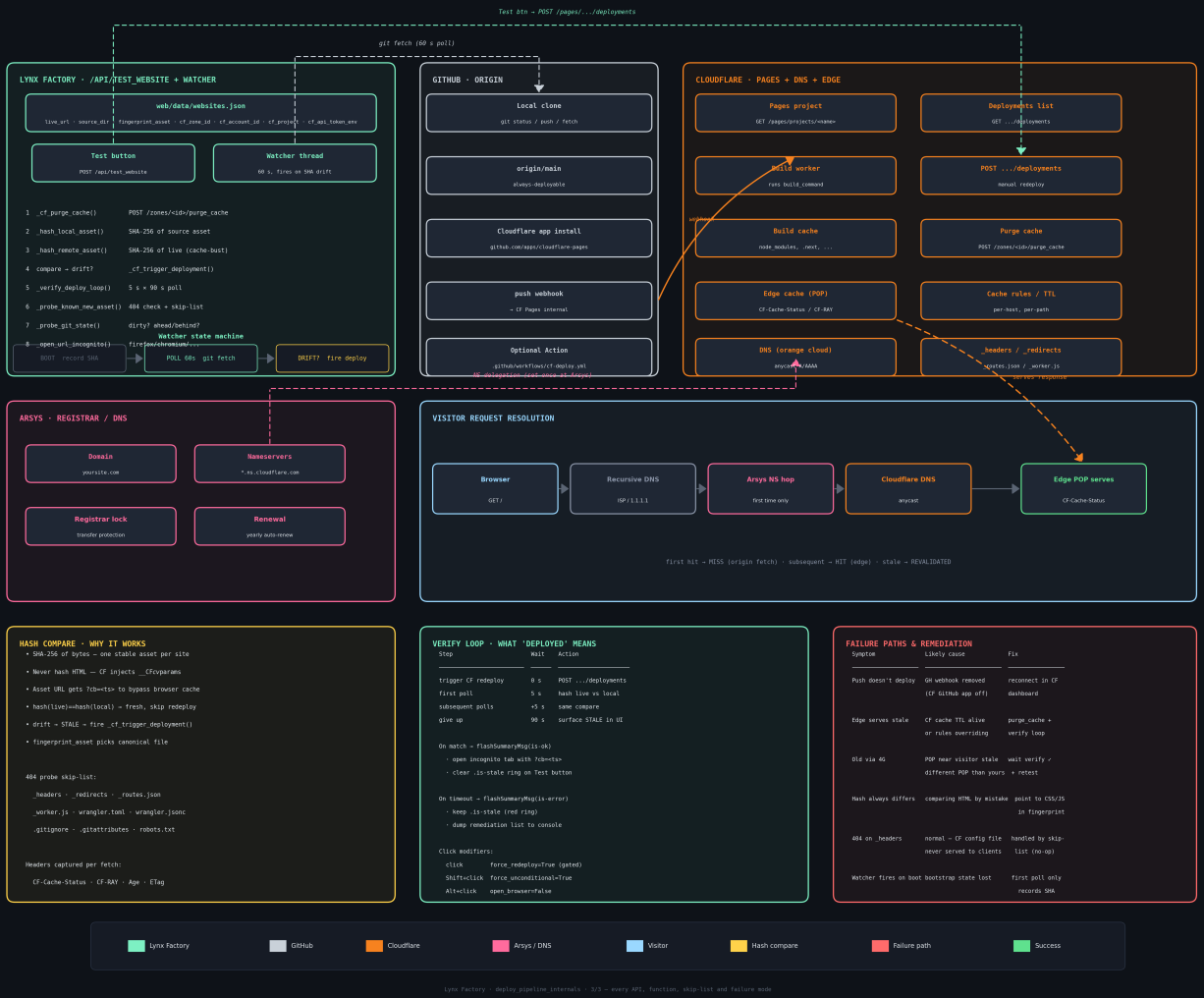
- Plain click → force_redeploy=True (still gated by staleness — won't trigger if everything matches).
- Shift + click → force_unconditional=True (always trigger a redeploy regardless of hash).
- Alt + click → open_browser=False (skip the incognito launch; useful for headless checks).

> **TIP**
> If the button turns red and stays red, the live site is stale. Hover for a tooltip; the browser console has the full remediation list.

Diagram 3 — Full internals (every API, function, skip-list, failure mode)

Website deploy pipeline — full internals

Every function, API endpoint, state transition and skip-list used by the Test button and the auto-redeploy watcher.



The auto-redeploy watcher

Lynx Factory runs a background thread that polls every 60 seconds. It compares the latest commit on origin/main against the last commit it saw. On drift, it dispatches a deploy automatically (same dispatcher as the Test button).

Why this matters

Cloudflare's GitHub auto-deploy webhook can silently disconnect (uninstalled app, revoked token, network blip). When it does, pushes look fine on your end but never produce a new deploy. The watcher is the safety net: if you pushed, Cloudflare will deploy — one way or another.

State machine

- BOOT → record current SHA, never fire (prevents 'phantom' deploys on every restart).
- POLL → every 60s: git fetch + git rev-parse origin/<branch>.
- DRIFT? → if remote SHA moved, call _dispatch_deploy(meta, ...).
- Idle → if no drift, loop back to POLL.

Where to see its state

```
$ curl -s http://127.0.0.1:9999/api/auto_redeploy_state | jq
{
  "watcher": "running",
  "interval_s": 60,
  "sites": {
    "galeriaomaso":   {
      "last_seen_sha":      "ad376ff...",
      "last_trigger_method": "wrangler",
      "last_fired_at":      "2026-05-17T13:42:11Z"
    }
  }
}
```

NOTE

The watcher requires per-project CF_API_TOKEN_<PROJECT> env vars and the cloudflare_account_id field in websites.json. Without them the watcher still polls but cannot fire a deploy.

Adding a new website (full recipe)

Canonical recipe for taking a brand-new domain and serving it through the same Lynx + GitHub + Cloudflare + Arsys pipeline.

1. Register the domain at Arsys

Buy it at clientes.arsys.es. Wait for the confirmation email.

2. Add the zone to Cloudflare

Dashboard → Add a site. Cloudflare gives you two nameservers.

3. Update Arsys nameservers

Switch to those two CF nameservers. Wait for propagation.

4. Create the GitHub repo

```
# on github.com: create empty repo 'mysite'

cd ~/projects
git clone git@github.com:you/mysite.git
cd mysite
echo '<h1>Hello world</h1>' > index.html
git add . && git commit -m 'first commit'
git push -u origin main
```

5. Create the Cloudflare Workers/Pages project

For static sites with wrangler: add a wrangler.jsonc in your public/ directory. First deploy via:

```
cd ~/projects/mysite/public
npx wrangler@latest deploy
```

Then wire the custom domain mysite.com + www in the CF dashboard.

6. Mint a scoped CF token for this project

Name it token-scoped-refresh-mysite. Scopes: Workers Scripts: Edit + Cache Purge: Purge, restricted to this account and zone only.

```
echo 'export CF_API_TOKEN_MYSITE="<scoped-token>"' \
  >> ~/.bashrc.d/lynx_factory_cf.sh
source ~/.bashrc.d/lynx_factory_cf.sh
```

7. Register the site with Lynx Factory

```
// edit web/data/websites.json — add an entry:
{
  "live_url":        "https://www.mysite.com",
  "source_dir":      "/home/you/projects/mysite",
  "public_subdir":   "public",
  "fingerprint_asset": "style.css",
  "cloudflare_zone_id":     "<zone-id>",
  "cloudflare_account_id":  "<account-id>",
  "cloudflare_project":     "mysite",
  "cloudflare_api_token_env": "CF_API_TOKEN_MYSITE",
  "deploy_method":          "wrangler",
  "auto_redeploy_on_push":  true,
  "auto_redeploy_branch":   "main"
}
```

8. Restart Lynx Factory and verify

```
pkill -f 'web/app.py'
nohup ./bin/lynx-factory --serve > /tmp/lynx-factory.log 2>&1 &

# open the multiplexer, find the new cell, click Test.
```

TIP

Pick a CSS/JS/PNG file as fingerprint_asset, not HTML. Cloudflare modifies HTML in flight (bot-management script), which breaks the SHA-256 compare.

Day-to-day workflow

Push a change to a live site

```
cd ~/projects/mysite
vim index.html
git add -p && git commit -m 'tweak headline'
git push                   # watcher dispatches deploy in <60s

# OR press Test in the multiplexer to deploy immediately.
```

Force a redeploy even if Lynx thinks nothing changed

In the multiplexer, Shift+click the Test button. This sets force_unconditional=True so a deploy is fired regardless of hash.

Check what the watcher is doing

```
curl -s http://127.0.0.1:9999/api/auto_redeploy_state | jq
```

Restart Lynx Factory cleanly

```
pkill -f 'web/app.py'
nohup ./bin/lynx-factory --serve > /tmp/lynx-factory.log 2>&1 &
disown
```

Tail Lynx Factory logs

```
tail -f /tmp/lynx-factory.log
```

Add a new env var (CF token, etc.)

```
echo 'export CF_API_TOKEN_NEWSITE="..."' \
  >> ~/.bashrc.d/lynx_factory_cf.sh
source ~/.bashrc.d/lynx_factory_cf.sh
# restart Lynx Factory so the Flask process inherits the new var
```

Roll back to a previous deployment

CF Dashboard → Workers & Pages → project → Deployments → pick an earlier successful deployment → Rollback. The edge cache flips back within seconds.

Where everything lives — file/code tour

Top-level layout

```
lynx_factory/
■■■ bin/
■   ■■■ lynx-factory               # launcher
■■■ web/
■   ■■■ __main__.py                # CLI entry: --serve / --browser
■   ■■■ app.py                     # Flask app + endpoints + watcher
■   ■■■ auth.py                    # @before_request auth gate
■   ■■■ data/
■   ■   ■■■ websites.json          # website metadata + CF config
■   ■   ■■■ logs.json              # rolling activity log
■   ■   ■■■ ...
■   ■■■ static/
■   ■   ■■■ multiplexer.js         # multiplexer UI + Test button
■   ■   ■■■ quota_panel.js         # week-bar + token-usage panel
■   ■   ■■■ style.css              # all visual styling
■   ■■■ templates/
■       ■■■ _base.html             # outer page chrome
■       ■■■ multiplexer.html       # cells with RST/EXEC/Test
■■■ docs/website_deploy/
■   ■■■ generate_infographics.py   # PNG diagram generator
■   ■■■ pdf_guide.py               # ReportLab PDF generator
■   ■■■ deploy_pipeline_*.{png,pdf} # generated outputs
■■■ prompts_in/  tracking/  steganography/
```

The Test button — where the code lives

- Server endpoint: web/app.py — /api/test_website.
- Dispatchers: _dispatch_deploy → _wrangler_deploy | _cf_trigger_deployment.
- Hash helpers: _hash_remote_asset, _hash_local_asset, _verify_deploy_loop.
- Client handler: web/static/multiplexer.js — checkFreshedWebsite.
- Button HTML: web/templates/multiplexer.html — mux-cell-btn-test.

The watcher thread

- _AUTO_REDEPLOY_THREAD, _AUTO_REDEPLOY_STATE, _auto_redeploy_tick — all in web/app.py.
- Started by start_background_threads() at app boot.
- Introspected via GET /api/auto_redeploy_state.

Why the Test button silently failed (and what changed)

Symptoms

Borja edited galeriaomaso.com's banner, committed and pushed to GitHub. The Cloudflare dashboard showed the GitHub push but no new deployment. The Test button in Lynx Factory reported success — yet the live site still served the old banner.

Root cause #1 — assumed Cloudflare GitHub auto-deploy was wired up

The project was assumed to be hooked to Cloudflare's GitHub integration ('push to main → CF builds → CF publishes'). It was not. Every prior deployment in the project's history had been done manually with the wrangler CLI. There was no GitHub App installed for this project — pushes were therefore invisible to Cloudflare.

Root cause #2 — wrong API endpoint for a Workers project

Lynx Factory's redeploy helper called the Cloudflare Pages REST API (POST /accounts/<id>/pages/projects/<name>/deployments). But galeriaomaso is a Workers project (with the static-assets bundler), not a Pages project. That endpoint returned HTTP 404 for every attempt — silently, because the response was logged but the Test button only reported the top-level OK flag.

Root cause #3 — no fingerprint hash check

Pre-existing Test code claimed success when the HTTP call to CF returned 2xx, regardless of whether anything actually shipped to the edge. There was no verification that the bytes on the live site had changed.

What the original Cloudflare-side behaviour was

Cloudflare was doing exactly what it was told: serve the most recently deployed Worker bundle from the edge cache, indefinitely. Because no fresh `wrangler deploy` had run since the previous successful manual deploy, the edge correctly served that old bundle. The GitHub push was ignored — there was no integration installed to react to it.

The fix

- Added _wrangler_deploy() in app.py: runs `npx wrangler@latest deploy` inside <source_dir>/<public_subdir>, with CF_API_TOKEN + CF_ACCOUNT_ID injected via env.
- Added _dispatch_deploy(): reads deploy_method from websites.json (wrangler | pages_api | github_auto) and routes to the right helper.
- Updated /api/test_website and _auto_redeploy_tick to call _dispatch_deploy instead of the Pages-only path.
- Added a fingerprint verify loop (5s × 90s) that re-hashes the live asset after deploy until it matches local.
- Switched LF to per-project least-privilege tokens (token-scoped-refresh-<project>).

> **TIP**
> After every deploy the Test button reports cf_redeploy.method (wrangler | pages_api | github_auto) and verify.elapsed_s. If method is wrangler and verify.ok is true, the new bytes are on the edge.

Emergency procedures

Live site is down

- Open Cloudflare → Workers & Pages → project → Deployments. Is the latest deployment red?
- If yes: click Rollback on the previous green deployment.
- If no: check CF → Overview. Is the zone status 'Active'?
- If both look fine: check dns.google or cachecheck.cloudflare.com for the domain.

Made a bad commit, already pushed

```
git revert HEAD --no-edit && git push
# the watcher will redeploy via wrangler in <60s
```

Cloudflare API token leaked

- Cloudflare → My Profile → API Tokens → click the token → Roll.
- Replace the value in ~/.bashrc.d/lynx_factory_cf.sh with the new one.
- source the file and restart Lynx Factory.
- Keep the same scope (per-project, least-privilege). Do not 'upgrade' to a broader token.

Cloudflare account locked out

- Recover via 2FA backup codes.
- If lost: cloudflare.com/recovery — requires identity verification, takes days.
- Meanwhile sites still serve — only the dashboard is locked, not the edge.

Arsys domain about to expire

- clientes.arsys.es → Dominios → Renovar.
- An expired domain takes the site down within hours. Set auto-renew + a yearly calendar reminder.

Lynx Factory will not start

```
tail -50 /tmp/lynx-factory.log
# common causes:
# - port 9999 already in use (kill the old python web/app.py)
# - websites.json malformed (python -m json.tool web/data/websites.json)
# - missing pip dependency (re-run the pip install line above)
```

DANGER

Before any destructive command, take a snapshot: cp -a web/data /tmp/lf-backup-$(date +%s) Lynx Factory keeps automatic backups under web/data/_backups/ but an extra one never hurts.

When things go wrong — troubleshooting

'I pushed but nothing changes on the live site'

- Click Test in Lynx Factory. If it turns red, the live hash differs from local — the redeploy didn't reach the edge.
- Open dash.cloudflare.com → Workers & Pages → your project → Deployments. Is there a recent deployment? Did it succeed?
- If there is none, the GitHub auto-deploy integration is either missing or broken — but Lynx's wrangler path should still work. Try Shift+click on Test to force unconditional redeploy.
- Check /tmp/lynx-factory.log for wrangler stderr.

'My phone shows the old site but my laptop shows the new one'

- Phone has a cached copy. Open in an incognito tab.
- Phone may be hitting a different CF edge POP. Wait a minute, retry.
- Home Wi-Fi may have a DNS cache returning an old answer. Reboot the router or try mobile data.

'Worked yesterday, broken today, I didn't change anything'

- Open Cloudflare → Deployments. Was a recent build red?
- If yes, the build log usually shows what changed (dependency, font, env).
- Fix and push again, or roll back to the previous successful deployment.

'The Test button is always red even after waiting'

- Make sure CF_API_TOKEN_<PROJECT> is exported and visible to the Flask process (restart LF after setting).
- Make sure cloudflare_account_id is set in web/data/websites.json.
- Confirm fingerprint_asset in websites.json points to a real file under source_dir (and is NOT HTML).
- Verify the scoped token actually has Workers Scripts: Edit on this account — call /accounts/<id>/tokens/verify with it.

DANGER

Never pick an HTML page as the fingerprint asset. Cloudflare injects a tiny JS snippet (__CFcvparams) into HTML responses on the fly, so the live HTML never matches the local HTML by SHA-256.

Architecture deep-dive

Process model

Lynx Factory is a single Python process. Flask runs in the main thread; several daemon threads run in the background — the relevant one here is the auto-redeploy watcher. There is no separate worker, no queue, no scheduler. Everything is in-process.

Data store

Configuration and state live as JSON files under web/data/. Reads are mtime-cached so hot paths do not re-parse the file on every request. Writes are atomic (write to .tmp, fsync, rename). Backups under web/data/_backups/ with rolling retention.

Auth model

web/auth.py installs a Flask @before_request hook that gates every endpoint unless the request carries a valid session cookie. The token lives at ~/.lynx/web-token; you visit /login?token=<paste> once per browser.

Cloudflare interaction model

- Read-only operations (GET project, GET deployments) use requests.get with a 10s timeout.
- Write operations (POST purge_cache, POST deployments) use requests.post with a 30s timeout.
- Wrangler deploys run via subprocess: npx --yes wrangler@latest deploy, with a 300s timeout.
- Tokens are read from env on every call (not cached) so rotating a token requires only a LF restart.

Why the verify loop polls instead of using webhooks

Cloudflare offers webhooks for deployment status but they require a publicly reachable URL — which your laptop is not. Polling every 5s for up to 90s is simpler, faster to debug, and good enough — a successful deploy usually reflects at the edge within 10-30 seconds.

Why hashing instead of trusting Cloudflare's status

A 'successful' build at Cloudflare does not always mean the edge serves the new content immediately. Cache rules, tiered cache, and regional propagation can lag. Comparing the actual bytes you receive against the bytes on disk is the only ground truth.

Glossary

- Cache: a temporary copy of a response kept close to the requester so the next request is faster.
- CDN (Content Delivery Network): a global fleet of edge servers each holding cached copies of your static files.
- CF-Cache-Status: a Cloudflare response header. HIT = served from edge cache, MISS = fetched from origin, REVALIDATED = origin confirmed cache is still fresh.
- Commit: a saved snapshot in git.
- DNS (Domain Name System): the system that turns yoursite.com into an IP address.
- Edge POP: a single Cloudflare data center, one of 200+ worldwide.
- Fingerprint asset: one stable file (CSS/JS/image — never HTML) used by the Test button to detect drift.
- Hash (SHA-256): a short fixed-length fingerprint of arbitrary bytes; two inputs produce the same hash only if they are byte-identical.
- Hook (webhook): an HTTP call one system makes to another when something interesting happens.
- Incognito tab: a browser window that bypasses your local cache and cookies.
- Least-privilege token: an API token with exactly the permissions the task needs and nothing more.
- main branch: the default 'always-deployable' branch on GitHub.
- Nameserver: a DNS server authoritative for a domain. Arsys delegates to Cloudflare's nameservers.
- Origin: the source-of-truth storage where the CDN fetches from when it does not have a cached copy.
- Pages (Cloudflare Pages): Cloudflare's static-site hosting product. Builds on push, serves at the edge.
- Push: uploading local git commits to a remote (GitHub).
- Purge: deleting a cached copy so the next request fetches a fresh one.
- Recursive resolver: the DNS server your machine actually queries (1.1.1.1, 8.8.8.8).
- Registrar: the company that holds the right to a domain name (Arsys here).
- Scoped token: a token whose Account / Zone Resources are restricted to a single account and zone.
- TTL (Time To Live): how long a cached answer is allowed to stay before being refreshed.
- Verify loop: the Test button's 5-second × 90-second polling that confirms a deploy has actually flushed to the edge.
- Watcher: Lynx Factory's background thread that auto-triggers a deploy when origin/main advances.
- Workers (Cloudflare Workers): Cloudflare's serverless platform. Lynx ships static-asset-bundle Workers via wrangler.
- wrangler: Cloudflare's CLI for managing Workers. Lynx invokes it via npx wrangler@latest deploy.

Quick reference — files, env vars, APIs

websites.json (per-project fields)

```
Required:
  live_url                 https://www.galeriaomaso.com

Recommended:
  source_dir               path to local repo
  public_subdir            subdir served at /   (default: public)
  fingerprint_asset        relative path (CSS/JS/image)
  cloudflare_zone_id       from CF dashboard URL
  cloudflare_account_id    from CF dashboard URL
  cloudflare_project       CF project name
  cloudflare_api_token_env e.g. CF_API_TOKEN_GALERIAOMASO
  deploy_method            wrangler | pages_api | github_auto
  auto_redeploy_on_push    true / false
  auto_redeploy_branch     main
```

Environment variables (~/.bashrc.d/lynx_factory_cf.sh)

```
CF_ACCOUNT_ID                one account ID for all sites
CLOUDFLARE_ACCOUNT_ID        alias of CF_ACCOUNT_ID (wrangler reads this)
CF_API_TOKEN_<PROJECT>       per-project scoped token
CF_ZONE_ID_<PROJECT>         per-project zone ID (for cache purge)
```

Cloudflare API endpoints used

```
POST /zones/<zone_id>/purge_cache
  body: {"purge_everything": true}

POST /accounts/<acc_id>/pages/projects/<name>/deployments
  used by deploy_method=pages_api  (Pages projects only)

wrangler subprocess (deploy_method=wrangler):
  npx --yes wrangler@latest deploy
    cwd: <source_dir>/<public_subdir>
    env: CLOUDFLARE_API_TOKEN + CLOUDFLARE_ACCOUNT_ID
```

Lynx Factory endpoints

```
POST /api/test_website
  body: {
    "name": "galeriaomaso",
    "force_redeploy":     true,    // default for Test btn
    "force_unconditional": false,  // Shift+click sets true
    "open_browser":       true     // Alt+click sets false
  }

GET  /api/auto_redeploy_state    # watcher introspection
```

Setup checklist (one-page summary)

On your laptop

- Mint one scoped CF token per project: token-scoped-refresh-<project>.
- Scopes: Workers Scripts: Edit + Cache Purge: Purge, restricted to this account and zone only.
- Export each as CF_API_TOKEN_<PROJECT> in ~/.bashrc.d/lynx_factory_cf.sh so it persists across reboots.
- Fill in cloudflare_zone_id, cloudflare_account_id, cloudflare_project, cloudflare_api_token_env, deploy_method per site in web/data/websites.json.
- Restart Lynx Factory so the new env vars and JSON are picked up.

Verify everything works

```
# token visible to current shell?
echo $CF_API_TOKEN_GALERIAOMASO | cut -c1-8

# Lynx watcher healthy?
curl -s http://127.0.0.1:9999/api/auto_redeploy_state | jq

# Click Test in the multiplexer.
# Mint-teal flash = OK · red = stale · console has the full report.
```

On Cloudflare

- (Optional) install the Cloudflare GitHub App for push-based auto-deploy. The wrangler path works without it.
- Confirm a recent deployment appears under Workers & Pages → project → Deployments.

On Arsys

- Confirm domain nameservers point at *.ns.cloudflare.com.
- Confirm domain renewal is set to auto-renew (or has a manual calendar reminder).

TIP

Once everything is configured you should rarely need to open the Cloudflare dashboard. Lynx's Test button + watcher covers the day-to-day deploy + verify loop.